

Documentación Técnica

Sistema de Gestión de Usuarios

1. Introducción

Este documento describe la solución desarrollada para la gestión de usuarios, incluyendo arquitectura, decisiones técnicas, base de datos y funcionalidades implementadas.

2. Arquitectura

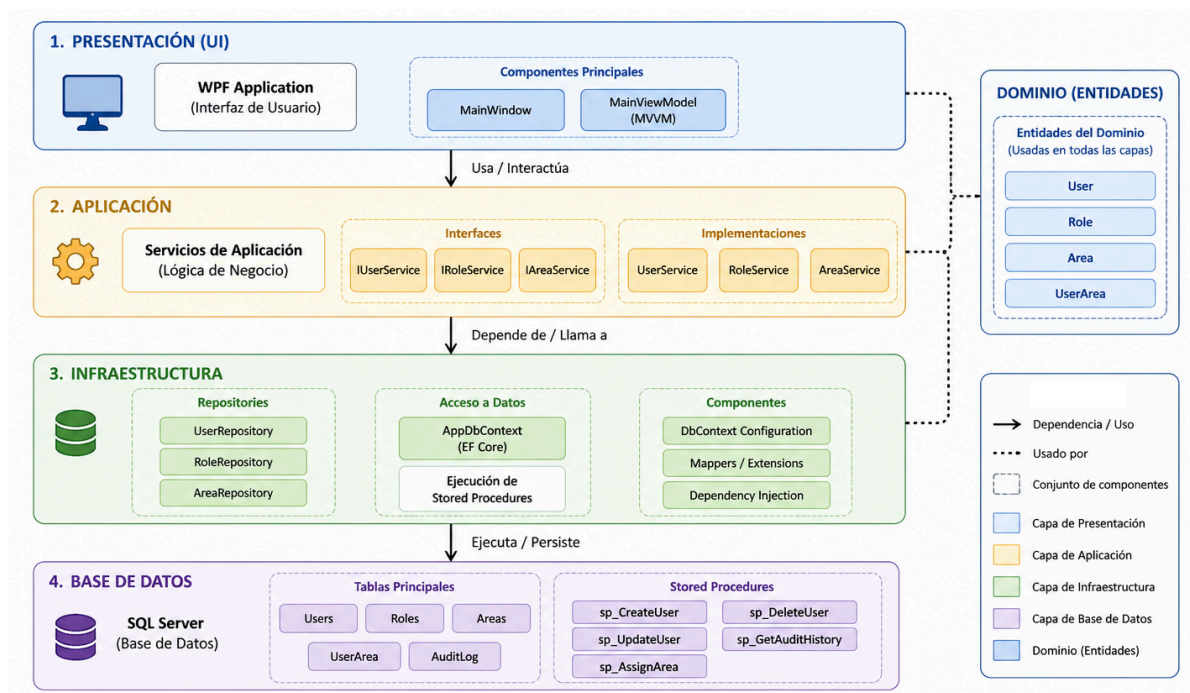
El sistema está diseñado bajo una arquitectura en capas:

- Presentation (WPF)
- Application (Servicios e Interfaces)
- Domain (Entidades)
- Infrastructure (Acceso a datos y SPs)

Se aplican principios SOLID y patrón MVVM.

3. Diagrama de Componentes

Figura 1. Diagrama de Componentes del Sistema



5. Tecnologías Utilizadas

- .NET 8
- WPF
- Entity Framework Core
- SQL Server
- MVVM
- Stored Procedures

6. Base de Datos

Se implementaron tablas: Users, Roles, Areas, UserArea, AuditLog.

Se utilizan stored procedures para:

- Crear usuario
- Actualizar usuario
- Asignar área
- Eliminar usuario
- Consultar auditoría

7. Auditoría

Se implementa auditoría basada en eventos de negocio:

- CREATE_USER
- UPDATE_USER
- ASSIGN_AREA
- DELETE_USER

Se almacenan OldValue y NewValue en formato JSON para trazabilidad completa.

Ejemplo:

CREATE_USER

```
{
  "Name": "Juan Perez",
  "ContactInfo": "123456789",
  "RoleId": 1
}
```

UPDATE_USER

OldValue:

```
{
  "Name": "Juan Perez",
  "ContactInfo": "123456789",
  "RoleId": 1
}
```

```
}
NewValue:
{
    "Name": "Juan Perez",
    "ContactInfo": "3126539786",
    "RoleId": 1
}
```

ASSIGN_AREA

```
{
    "UserId": 3,
    "AreaId": 3,
    "AreaName": "Facturacion"
}
```

DELETE_USER

OldValue:

```
{
    "Id": 3,
    "Name": "Juan Perez",
    "ContactInfo": "3126539786",
    "RoleId": 1,
    "Status": true,
    "CreatedAt": "2026-05-03T11:01:52.7787201",
    "UpdatedAt": "2026-05-03T11:02:07.5606432"
}
```

NewValue:

```
{
    "Id": 3,
    "Name": "Juan Perez",
    "ContactInfo": "3126539786",
    "RoleId": 1,
    "Status": false,
    "CreatedAt": "2026-05-03T11:01:52.7787201",
    "UpdatedAt": "2026-05-03T11:02:28.3387307"
}
```

8. Funcionalidades

- CRUD de usuarios
- Asignación de áreas
- Visualización en DataGrid
- Soft delete
- Auditoría completa

9. Decisiones Técnicas

- Uso de Soft Delete para preservar historial
- Uso de Stored Procedures para control transaccional
- Separación de responsabilidades por capas
- Uso de DTOs para desacoplar UI y dominio

11. Repositorio del Proyecto

GitHub: <https://github.com/rafredyamirez/kibernum-backend>

12. Ejecución del Proyecto

1. Ejecutar scripts de base de datos
2. Configurar cadena de conexión
3. Ejecutar proyecto WPF

13. Conclusión

Se incorporan buenas prácticas de arquitectura y diseño de software que garantizan su mantenibilidad y escalabilidad. A través de una arquitectura en capas y el uso del patrón MVVM, se logró una clara separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y el acceso a datos. Asimismo, la implementación de stored procedures permitió centralizar la lógica crítica en la base de datos, asegurando consistencia transaccional y control sobre las operaciones.

Adicionalmente, se diseñó un sistema de auditoría robusto basado en eventos de negocio, utilizando estructuras JSON para almacenar el estado previo y posterior de cada operación, lo que facilita la trazabilidad y el análisis histórico de los datos. La utilización de soft delete refuerza la integridad de la información y permite conservar el historial sin afectar las relaciones existentes.